MCS-219: Object Oriented Analysis and Design

Assignment MCA_NEW/MCAOL(II)/219/Assign/2026-27 (Simple Answers)

Name: _____ Enrolment No.: _____

Q1a. OO paradigm vs Structured Analysis and Design

Philosophy of OO: The system is seen as a set of **objects**. Each object keeps its own **data (attributes)** and **behaviour (methods)** together. Objects work by sending messages to each other. Models follow real-world things (Customer, Account, Order), so the design is close to the problem.

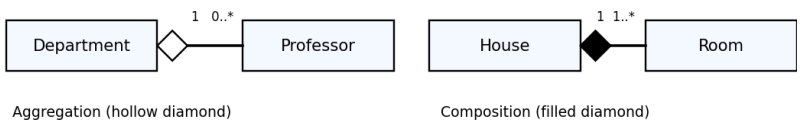
Structured approach: The system is seen as a set of **functions/processes** that change data. Data and functions are kept separate (DFDs, structure charts, top-down decomposition).

Point	Structured (SA/SD)	Object-Oriented
Data-control coupling	Functions pass and share data (often global data or flags). A change in data format affects many functions: high coupling.	Data is private to the object and accessed only through its methods. Objects talk through small interfaces: low coupling.
Computational encapsulation	Not supported. Data and procedures are separate, so any function can touch the data.	Data and the operations on it are packaged inside the class and hidden from outside.
Maintainability	A change in requirements ripples through many modules. Reuse is low.	Changes stay inside a class. Inheritance and polymorphism give reuse and easy extension. Easier to maintain in the long run.

Q1b. Four pillars of object orientation

Pillar	Meaning	Example
Abstraction	Show only the essential features, hide unnecessary details.	A *BankAccount* class shows deposit(), withdraw(), getBalance(). Users do not see how the balance is stored.
Encapsulation	Bind data and methods in one unit and hide the data (private); access only through public methods.	*balance* is private. withdraw() checks that balance is enough before reducing it.
Inheritance	A child class gets the attributes and methods of a parent class (reuse, is-a).	*SavingsAccount* and *CurrentAccount* inherit from *Account*. Each adds its own features.
Polymorphism	The same message gives different behaviour depending on the object.	calculateInterest() works differently in SavingsAccount and LoanAccount, but the caller uses the same call.

Q2a. Aggregation vs Composition



Aggregation and composition notation

Point	Aggregation	Composition
Meaning	Weak has-a. Whole has parts, but parts can exist alone.	Strong part-of. Parts belong to only one whole.
Lifecycle	Part is independent. If the whole is deleted, parts remain.	Part lives and dies with the whole. If the whole is deleted, parts are deleted.
Sharing	Part can be shared by many wholes.	Part cannot be shared.
UML	Hollow diamond	Filled diamond
Example	A Department has Professors. If the department closes, the professors still exist.	A House has Rooms, or an Order has OrderLines. Delete the order and the lines go too.

Q2b. Abstract class vs Interface

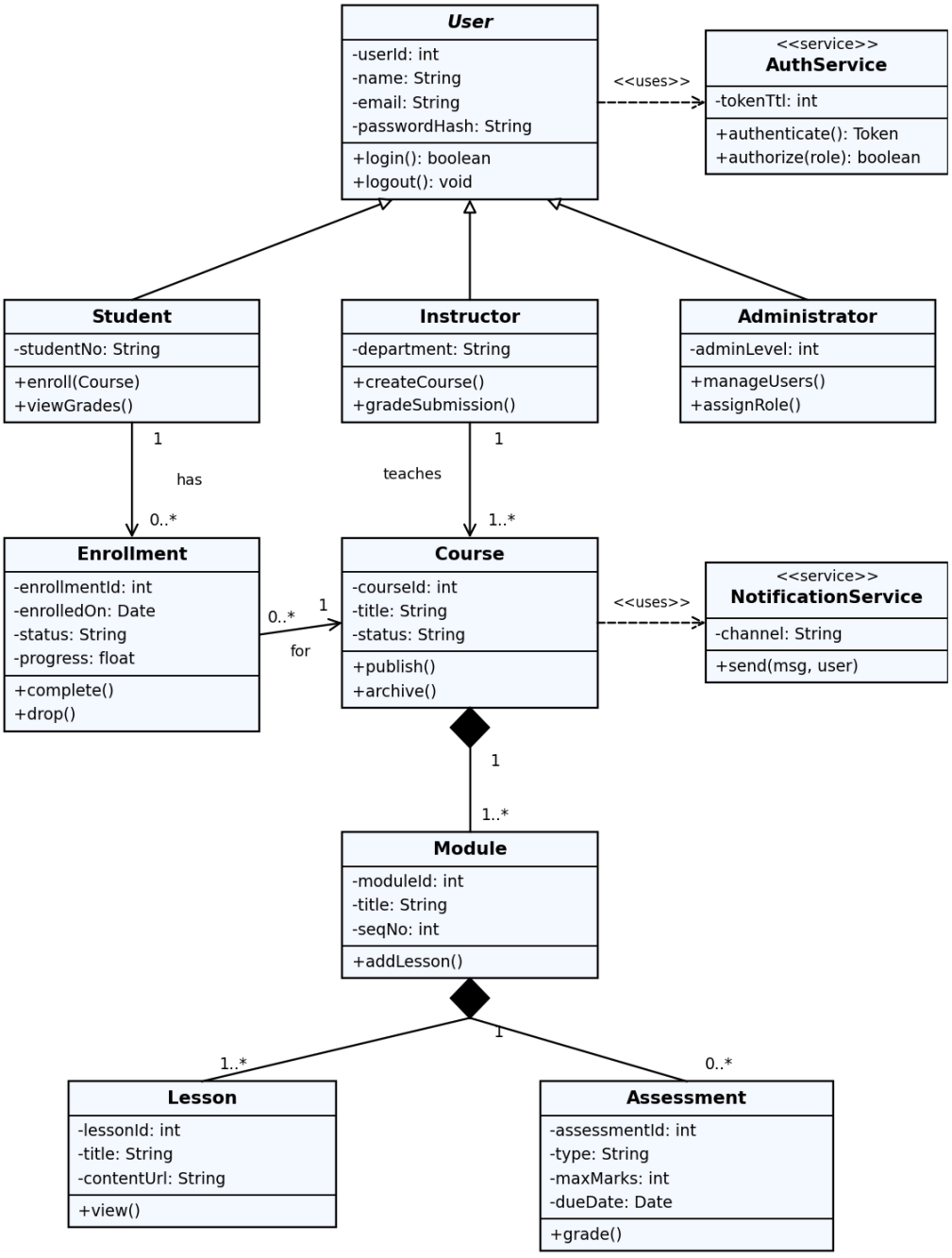
Point	Abstract class	Interface
Purpose	Partly built base class that shares common code and state for related classes.	Pure contract: only what a class can do (method signatures). Any class can implement it.
Members	Can have fields, constructors, concrete and abstract methods.	Mostly method declarations (and constants). No state.
Inheritance	A class can extend only one (in Java/C#).	A class can implement many interfaces.
Use when	Classes are closely related (is-a) and need shared code. Example: Shape with common colour field.	Unrelated classes need the same ability. Example: Payable, Printable, Comparable.

Trade-offs: An interface gives **loose coupling** and flexibility (many implementations, easy testing and mocking), but it cannot share code, and adding a method later breaks all implementers. An abstract class gives **code reuse**, but it locks the design into one parent and creates tighter coupling. **Rule:** use an interface for decoupling boundaries between subsystems; use an abstract class when subclasses really share state and behaviour.

Q3. Class Diagram: Learning Management System (LMS)

Assumptions:

- A user is either a Student, an Instructor or an Administrator (generalization from abstract class User).
- Authentication (login, role check) is handled by a separate AuthService (cloud-native, token based). User depends on it.
- A Course is owned by one Instructor; an Instructor may teach many courses.
- A Course contains Modules; a Module contains Lessons and Assessments (composition: they cannot exist without their parent).
- A Student takes many Courses and a Course has many Students. This many-to-many link is modelled by the **Enrollment** class, which stores status and progress (enrollment is recorded asynchronously, and NotificationService informs the user).
- Visibility: + public, - private, # protected.

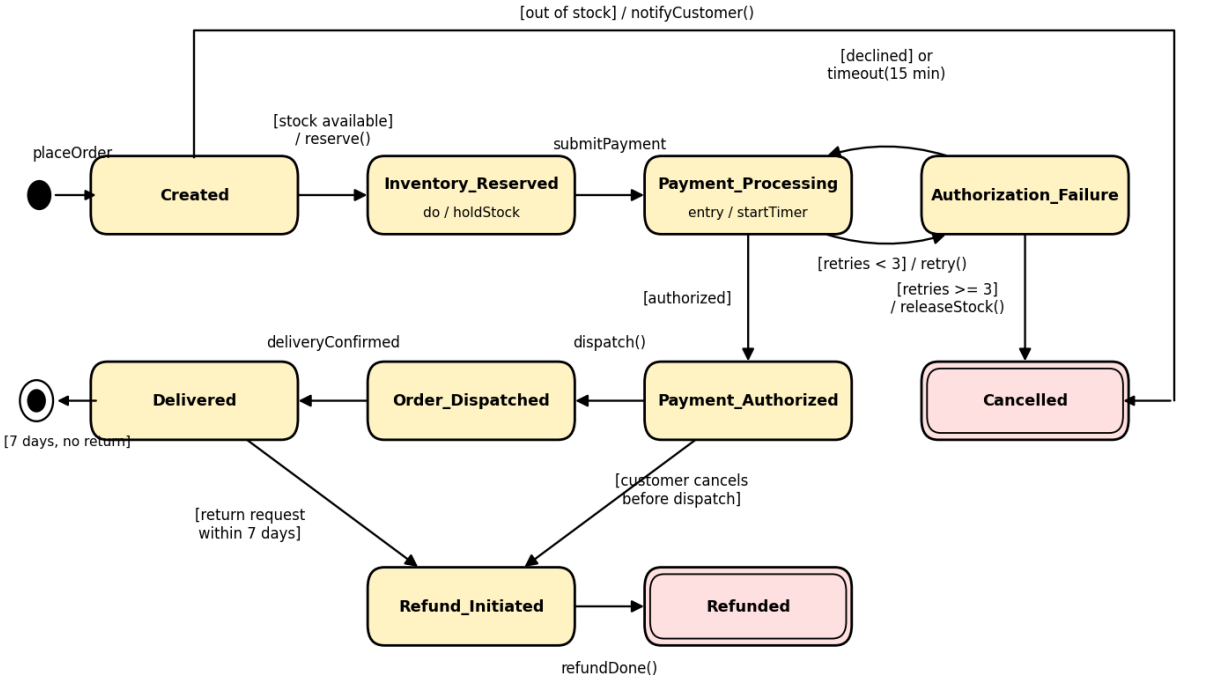


Class diagram of LMS

Q4. State Machine Diagram: Order lifecycle in e-Commerce

Assumptions:

- Payment is retried up to 3 times; a payment attempt times out after 15 minutes.
- A customer can cancel only before dispatch; a return can be requested within 7 days of delivery.
- Stock is held (reserved) before payment and released if the order is cancelled.



State machine of an order

Transition	Event	Guard	Action
Created to Inventory_Reserved	placeOrder / stock check	[stock available]	reserve()
Created to Cancelled	stock check	[out of stock]	notifyCustomer()
Inventory_Reserved to Payment_Processing	submitPayment		start timer
Payment_Processing to Payment_Authorized	gateway reply	[authorized]	
Payment_Processing to Authorization_Failure	gateway reply / timeout	[declined] or [timeout]	
Authorization_Failure to Payment_Processing	retry	[retries < 3]	retry()
Authorization_Failure to Cancelled		[retries >= 3]	releaseStock()
Payment_Authorized to Order_Dispatched	dispatch		create shipment
Payment_Authorized to Refund_Initiated	cancelRequest	[before dispatch]	start refund
Order_Dispatched to Delivered	deliveryConfirmed		
Delivered to Refund_Initiated	returnRequest	[within 7 days]	start refund
Refund_Initiated to Refunded	refundDone		notify customer

Q5. Object, Dynamic and Functional models (ATM example)

Good software analysis looks at a system from three views. Each view answers a different question and the three work together (as in the OMT method).

Model	Question it answers	ATM example	Notation
Object model (structural)	**What** exists in the system and how things are related?	Classes: ATM, Card, Customer, Account, Bank, Transaction, CashDispenser. Attributes: cardNo, balance. Relations: Customer owns Card; Card linked to Account; ATM connected to Bank.	Class diagram
Dynamic model (behavioural)	**When** do things happen? Order of events and states over time.	States: Idle, CardInserted, PINEntry, Menu, Processing, DispensingCash, EjectCard. Events: insertCard, enterPIN, selectWithdraw, timeout. 3 wrong PINs: card retained.	State diagram, sequence diagram
Functional model (dataflow)	**How** is data transformed? What goes in and out?	Processes: verify PIN, check balance, update account, dispense cash, print receipt. Data: PIN, amount, balance, receipt. Data stores: Account file.	Data flow diagram (DFD)

Unique, non-overlapping roles: The object model captures the *static structure* (who), the dynamic model captures the *control and timing* (when), and the functional model captures the *computation and data transformation* (what is calculated).

Interplay: Events in the dynamic model trigger operations of classes in the object model. Each operation is a process in the functional model, and the data it uses are attributes of objects. For example, the event *enterPIN* (dynamic) calls *verifyPIN()* of Card (object), which is the process *verify PIN* (functional) that reads the PIN and the stored card data.

Q6a. Synchronization in UML

Synchronization means coordinating concurrent activities and message exchanges. UML supports it through:

- **Synchronous messages** (sequence diagram): solid line with filled arrowhead. The sender **waits** until the receiver finishes and returns a reply (dashed arrow).
- **Asynchronous messages:** open arrowhead. The sender continues without waiting.
- **Fork and join (synchronization bars):** in activity and state diagrams, a **fork** splits one flow into concurrent flows; a **join** waits until all incoming flows finish before moving on.
- **Concurrent states:** a composite state can have parallel regions. They synchronize through events or by joining.
- **Operation concurrency property:** an operation can be marked {sequential} (one caller at a time), {guarded} (calls are made one at a time using a lock, others wait) or {concurrent} (many calls at once safely).
- **Activation bars** show how long an object is busy, and guards [condition] control when a transition happens.

Q6b. Tasks performed during system design

1. Partitioning into subsystems: Break the big (monolithic) system into smaller, loosely coupled subsystems with high cohesion, each with a clear interface. Methods: layers (UI, business, data) and partitions (peers, e.g. Order, Payment, Inventory). Benefits: parallel work, easy change, independent deployment.

2. Concurrency and process allocation: Find objects that must work at the same time (inherent concurrency, seen in the dynamic model) and give them separate threads or processes. Objects that

are never active together can share one thread. Decide which subsystems go to which processors or servers, and how they communicate (messages, queues, RPC).

3. Global resource allocation and data-store management: Decide how shared resources (CPU, memory, devices, database connections, files) are shared and who controls them, using priorities and locking. Choose the data store type: relational database, files, or object/NoSQL database, depending on volume, speed and transactions.

4. Boundary conditions: Handle the special cases at the edges of the system. *Initialization:* start-up, load configuration, create objects. *Termination:* release resources, save data, close connections. *Failure/recovery:* handle crashes, power or network failure with logging, backups, retry and rollback.

Q7. Mapping LMS class model to a 3NF relational schema

Mapping patterns used:

- Each class becomes a **table**; each attribute becomes a **column**. Every object gets a surrogate **primary key** (object identity).
- **One-to-many** association (Instructor-Course, Course-Module): put a **foreign key** on the 'many' side.
- **Many-to-many** (Student-Course): create a **junction table**. Here the association class **Enrollment** becomes the junction table with its own attributes.
- **Composition** (Course-Module-Lesson): foreign key with **ON DELETE CASCADE**.
- **Inheritance** (User, Student, Instructor, Administrator): **Table-Per-Class (joined)** strategy. A USERS table holds common columns. Each subclass table has a PK that is also an FK to USERS.
- Why not Single-Table? It would put many NULL columns (studentNo, department, adminLevel) in one table and break 3NF. Table-Per-Class avoids NULLs and duplication, at the cost of a join.

Table	Columns (PK underlined by name, FK marked)	Notes
USERS	**user_id PK**, name, email (UNIQUE), password_hash, role	Base table for User
STUDENTS	**user_id PK, FK to USERS**, student_no	Subclass
INSTRUCTORS	**user_id PK, FK to USERS**, department	Subclass
ADMINISTRATORS	**user_id PK, FK to USERS**, admin_level	Subclass
COURSES	**course_id PK**, title, status, **instructor_id FK to INSTRUCTORS**	1 instructor : many courses
MODULES	**module_id PK**, title, seq_no, **course_id FK to COURSES**	Cascade delete
LESSONS	**lesson_id PK**, title, content_url, **module_id FK to MODULES**	Cascade delete
ASSESSMENTS	**assessment_id PK**, type, max_marks, due_date, **module_id FK to MODULES**	Cascade delete
ENROLLMENTS	**enrollment_id PK**, **student_id FK to STUDENTS**, **course_id FK to COURSES**, enrolled_on, status, progress. UNIQUE(student_id, course_id)	Resolves M:N

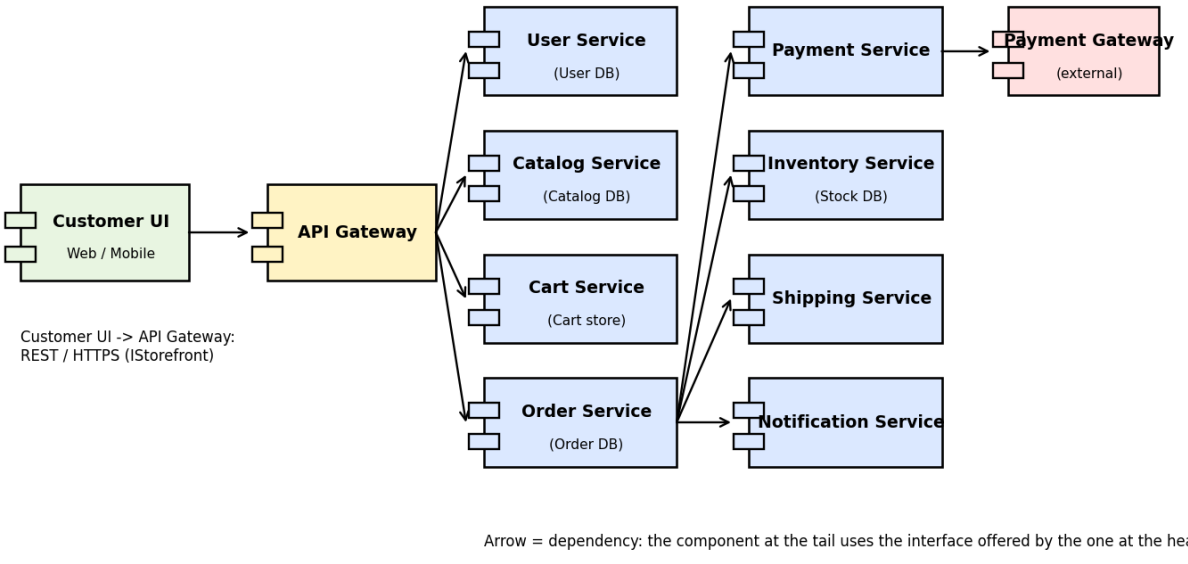
```
CREATE TABLE enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT NOT NULL REFERENCES students(user_id),
  course_id INT NOT NULL REFERENCES courses(course_id),
  enrolled_on DATE, status VARCHAR(20), progress DECIMAL(5,2),
  UNIQUE (student_id, course_id)
);
```

3NF check: every table has atomic columns (1NF), each non-key column depends on the whole key (2NF), and no non-key column depends on another non-key column (3NF). Instructor details are stored once in INSTRUCTORS, not repeated in COURSES. **Referential integrity** is kept by the FK constraints: a course cannot point to a missing instructor and an enrollment cannot point to a missing student or course.

Q8a. Hurdles in converting UML to code (C++, Java, C#)

- **Associations:** Languages have no direct 'association'. It must be coded with references, pointers or collections (List, Set). Keeping both ends of a **bidirectional** association consistent needs extra code.
- **Multiplicity:** Limits like 1..* or 0..1 cannot be written directly; they must be enforced with checks in constructors and setters.
- **Encapsulation:** Exposing a collection through a getter lets outsiders change it and breaks encapsulation. Fix: return read-only or copied views. Plain getters/setters for every field also leak internals.
- **Visibility mismatch:** UML's + - # (and ~ package) do not map one-to-one: C++ has friend, Java has package-private, C# has internal.
- **Multiple inheritance:** UML allows it, but Java and C# do not (use interfaces). C++ allows it but brings the diamond problem.
- **Composition and lifetime:** 'Part dies with whole' needs careful object creation and deletion (destructors in C++); in Java/C# the garbage collector decides.
- **Association classes, constraints, derived attributes and OCL** have no language support and must be hand-coded.
- **Keeping model and code in sync:** Code changes quickly and diagrams become outdated, so round-trip engineering tools are needed.

Q8b. Component Diagram: Online Shopping System



Component diagram of an online shopping system

Explanation: The Customer UI talks to the API Gateway, which routes requests to User, Catalog, Cart and Order services. The Order Service calls Payment, Inventory, Shipping and Notification services. The Payment Service uses an external Payment Gateway. Each service is an independent component with its own interface and its own data store.